



Programmable Platforms

Last reviewed: 7 months ago

Introduction

A programmable platform allows customers to customize a product by writing code. Unlike traditional SaaS with fixed features, it enables users to extend functionality, deploy backend logic, and build full-stack experiences—all within the platform's infrastructure.

Hosting the infrastructure for these platforms presents several challenges, including security, scalability, cost efficiency, and performance isolation. Allowing

customers to run custom code introduces risks such as untrusted execution, potential abuse, and resource contention, all of which must be managed without compromising platform reliability. Running millions of single-tenant applications is inherently costly, making efficient resource utilization critical. The ability to scale workloads to zero when idle is key to ensuring economic viability while maintaining rapid startup times when demand spikes. Additionally, ensuring seamless global execution with low-latency performance requires a resilient, distributed architecture. Robust monitoring, debugging, and governance capabilities are also essential to provide visibility and control over customer-deployed code without restricting innovation.

[Workers for Platforms](#) provides the ideal infrastructure for building programmable platforms by offering secure, isolated environments where customers can safely execute custom code at scale, with automatic scaling to zero and a globally distributed runtime that optimizes performance and cost.

Core Architecture Components

The Workers for Platforms architecture consists of several key components that work together to provide a secure, scalable, and efficient solution for multi-tenant applications. In the following core concepts are outlined.

1. **Main Request Flow:** An overview over the a request flow in a programmable platform.
2. **Invocation & Metadata Flow:** commonly, incoming requests and enriched with metadata to provide the function invocation with relevant context or perform routing logic.
3. **Egress Control:** controlling outbound connections to ensure compliant behaviour.
4. **Utilizing Storage & Data Resources:** leveraging databases & storage to build even richer end-user experiences at scale.

5. **Observability Tools:** Logging and metrics collection services to monitor platform performance and troubleshoot issues.

Main Request Flow

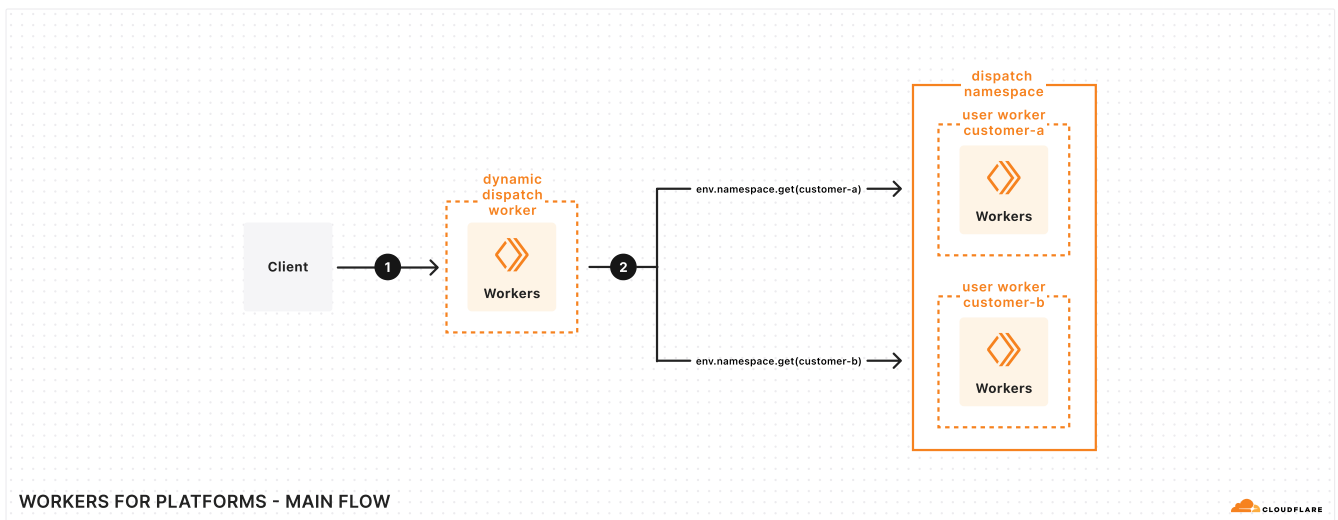


Figure 1: Workers for Platforms: Main Flow

1. **Client Request:** Send request from a client application to the platform's [Dynamic Dispatch Worker](#).
2. **Routing:** Identify the correct workload to execute and route the request to the respective [User Worker](#) in the [Dispatch Namespace](#). Each customer's workload runs in an isolated User

Worker with its own resources and security boundaries.

Invocation & Metadata Flow

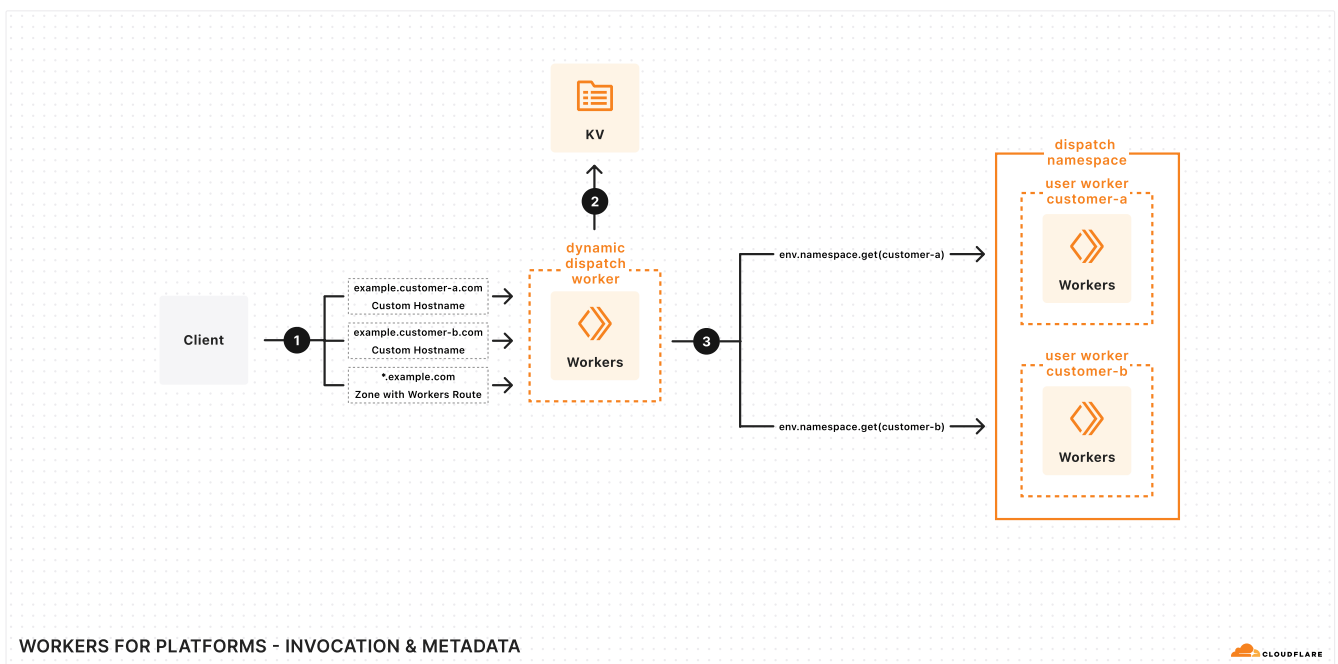


Figure 2: Workers for Platforms: Main Flow

For many use cases, it makes sense to retrieve additional metadata, user data, or configuration to process incoming requests and provide the User Worker invocation with additional context.

1. **Incoming Request:** Send requests to custom hostnames or a Worker using a Workers wildcard route.
2. **Metadata Lookup:** Retrieve customer-specific configuration data from [KV](#) storage. These lookups are typically based on the hostname of the incoming request or custom metadata in the case of custom hostnames.
3. **Worker Invocation:** Route requests to the appropriate User Worker in the Dispatch Namespace based on metadata. Optionally, provide additional context during function invocation.

Egress Control Pattern

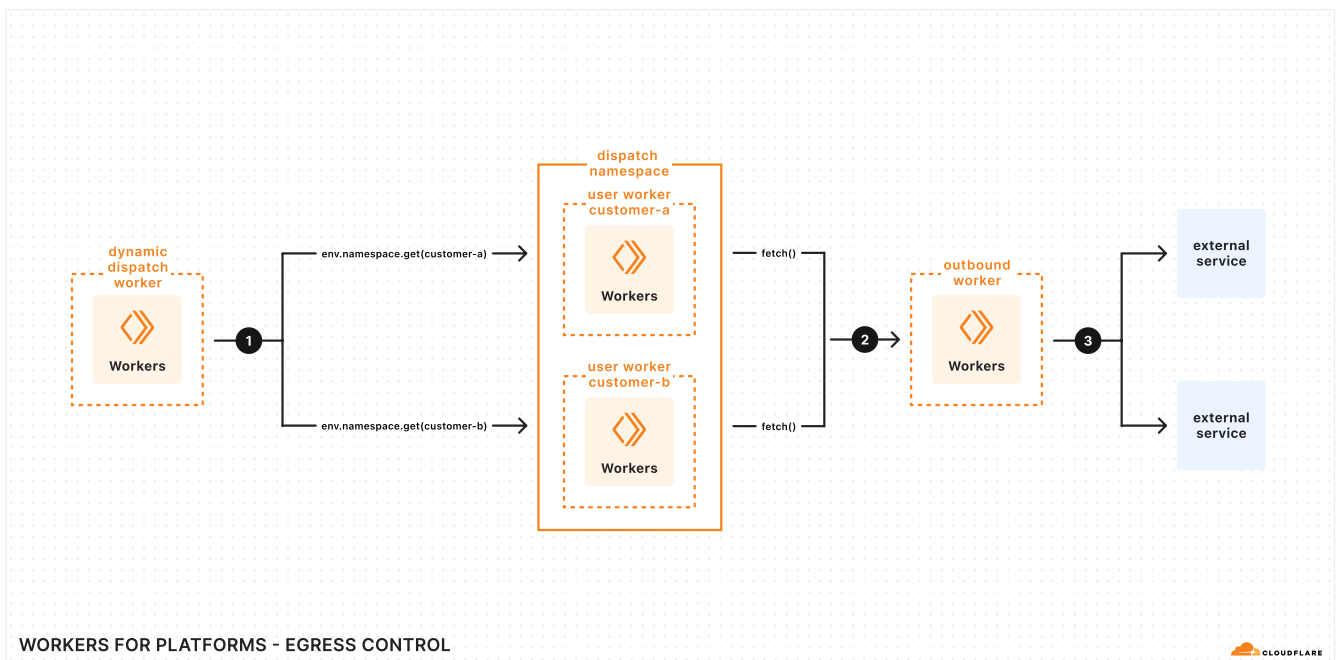


Figure 3: Workers for Platforms: Egress Control

Data observability and control is crucial for security. [Outbound Workers](#) allow for interception of all outgoing requests in User Worker scripts.

1. **Worker Invocation:** Route requests to the appropriate User Worker in the Dispatch Namespace. Optionally pass additional parameters to the Outbound Worker during User Worker invocation.
2. **External requests:** Send requests via `fetch()` calls to external services through a controlled Outbound Worker.

3. **Request interception:** Evaluate outgoing requests and perform core functions like centralized policy enforcement and audit logging.

Metrics & Logging Architecture

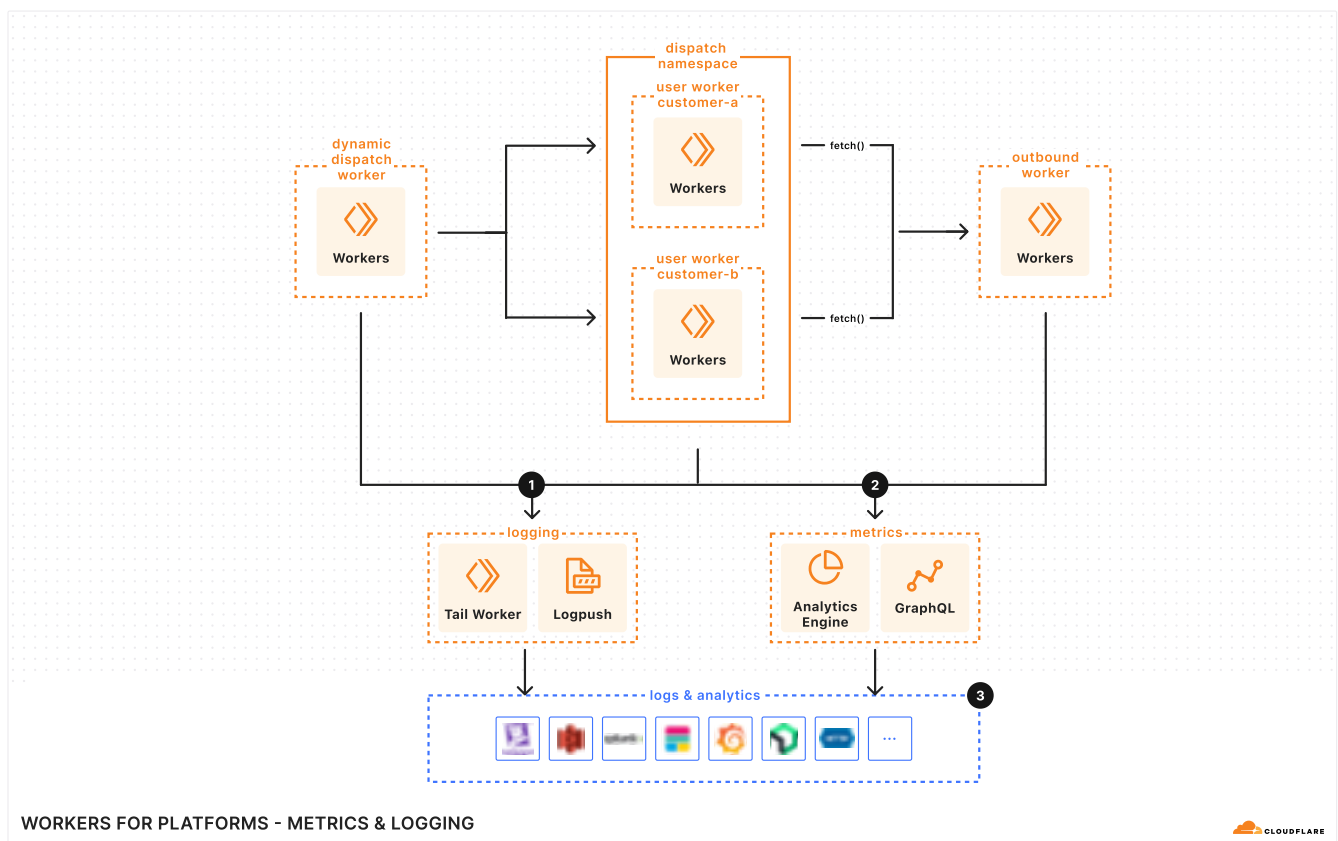


Figure 4: Workers for Platforms: Metrics & Logging

1. **Logging:** Collect logs throughout all Workers in the request flow via [Tail Worker](#) and [Workers Trace Events Logpush](#) services.

2. **Metrics:** Collect custom metrics via [Workers Analytics Engine](#) and out-of-the-box [Analytics](#) that can readily be queried via GraphQL API.
3. **Third-party Integration:** Export logs and metrics to various external monitoring and analytics platforms like Datadog, Splunk, Grafana, and others via [Analytics integrations](#).

Resource Isolation Model

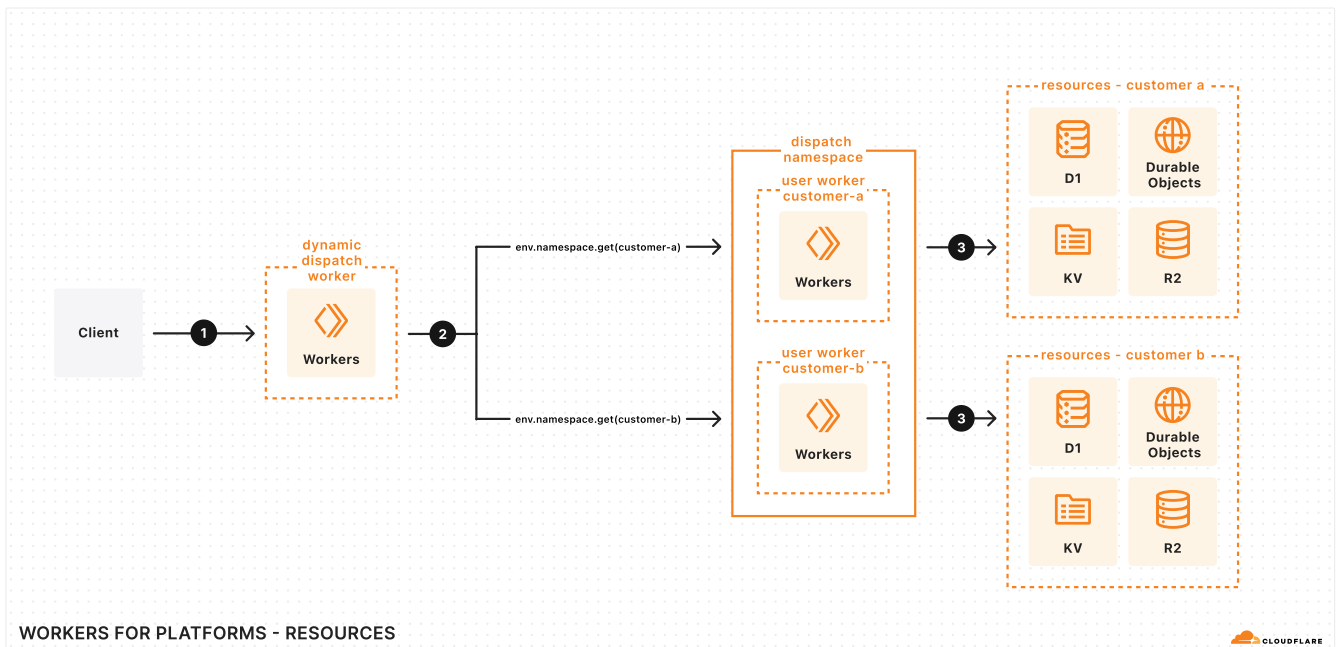


Figure 5: Workers for Platforms: Resources

1. **Incoming Request:** Send requests to custom hostnames or a Worker using a Workers wildcard route.
2. **Worker Invocation:** Route requests to the appropriate User Worker in the Dispatch Namespace.
3. **Resource Access:** Interact with per-script-specific resources:
 - D1 for relational database storage
 - Durable Objects for strongly consistent data
 - KV for high-read, eventually consistent key-value storage
 - R2 for object storage

Deployment & Management Flow

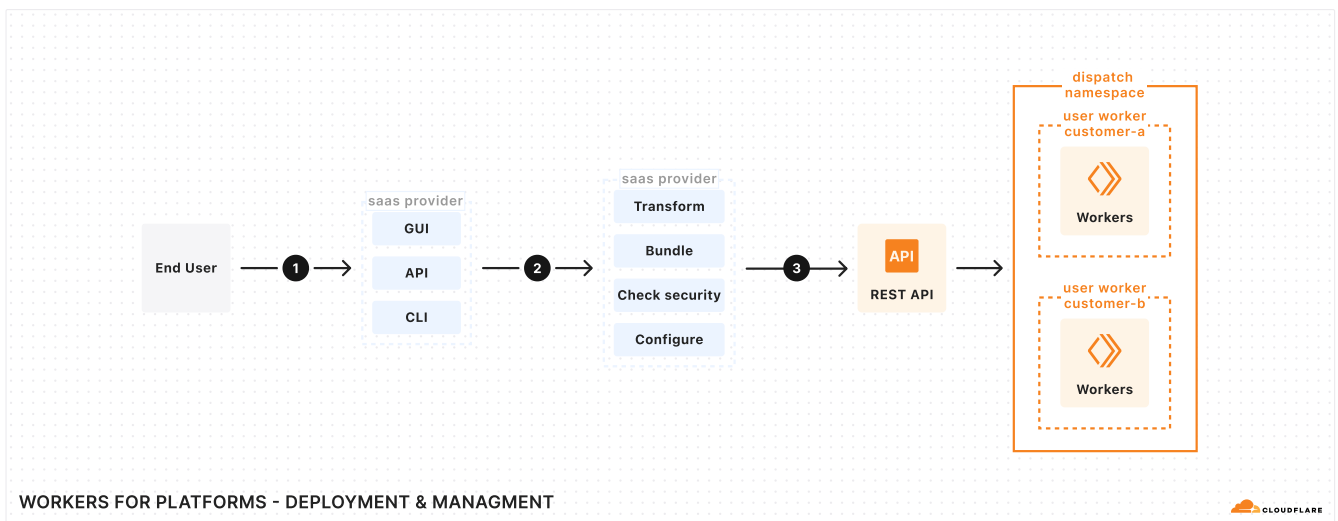


Figure 6: Workers for Platforms: Deployment & Management Flow

- 1. Management Interface:** Interact with the platform through GUI, API, or CLI interfaces.
- 2. Platform Processing:** Process these interactions to:
 - Transform and bundle code
 - Perform security checks
 - Apply configuration
- 3. Change Management:** Deploy changes to Cloudflare using the Cloudflare REST API.

Conclusion

Cloudflare Workers for Platforms provides a robust foundation for building multi-tenant SaaS applications with strong isolation, global distribution, and scalable performance. By leveraging this architecture, platform providers can focus on delivering value to their customers while Cloudflare handles the underlying infrastructure complexity.

Related resources

- [Workers for Platforms: Get started](#)
- [Workers for Platforms: Outbound Workers](#)
- [Workers for Platforms: Observability](#)

Was this helpful?



[Previous](#)[Next](#)[← Fullstack applications](#)[Serverless ETL pipelines →](#)

Last updated: Aug 19, 2025

Resources

[API](#)[New to Cloudflare?](#)[Directory](#)[Sponsorships](#)[Open Source](#)

Support

[Help Center](#)[System Status](#)[Compliance](#)[GDPR](#)

Company

[cloudflare.com](#)[Our team](#)[Careers](#)

Tools

[Cloudflare Radar](#)[Speed Test](#)[Is BGP Safe Yet?](#)[RPKI Toolkit](#)[Certificate Transparency](#)

Community



 [Discord](#)

 [YouTube](#)

 [GitHub](#)

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)